

UPPMAX Instruction

for Language Technology Students

03.07.2026

Uppsala Universitet
Department of Linguistics and Philology

Contents

Acknowledgement	1
1. Introduction	2
1.1. Working with Linux	2
1.1.1. Terminal, Console, Shell and CLI	2
1.1.2. Users, Groups and Permission	3
1.1.3. Editors	5
1.2. UPPMAX	6
1.3. The new cluster: Pelle	7
1.3.1. What's New	7
1.3.2. Hardwares	7
2. Moving around	8
2.1. Login to Pelle	8
2.1.1. Setting up TOTP	8
2.1.2. SSH key	8
2.2. Projects on UPPMAX	10
3. Files Transfer and Synchronisation	11
3.1. scp	11
3.2. rsync	11
4. Softwares	13
4.1. Module System	13
4.2. Installing Binary	13
4.3. Compiling	13
5. Python Virtual Environment	15
5.1. python-venv	15
5.2. conda	15
5.3. uv	15
5.3.1. Installation	15
5.3.2. Using venv	15
5.3.3. Packages	16
6. Scheduling Jobs on Pelle	17
6.1. Nodes	17
6.2. Interactive Session	17
6.3. Slurm	17
6.4. Time Management for Jobs	17
7. Storage Management	18
7.1. Available Storages	18
7.2. Storage Usage	18
7.3. Cache	18
8. Using Jupyter Lab or Notebook	19
8.1. UPPMAX Lab	19
8.2. Running on an Interactive Session	19
8.2.1. Request an interactive session	19
8.2.2. Run Notebook in the interactive session	19
8.2.3. SSH forward	19
9. Git	21
9.1. Forges	21

Acknowledgement

TBD

1. Introduction

Example: Tip & Note

Blocks in normal colours are general tips or notes. Commands under these blocks should be safe to run.

Example: Warning

Blocks in red and yellow are marked as *warning*, which will include things that you might need to pay special attention to. These things might be dangerous, regarding personal data, or information security.

Command Formats

I will use `$` to indicate a command, unless it is embedded in the text, which should be available for both Bash and PowerShell. If a command is only valid on Windows, it will be `PS>` to indicate PowerShell.

1.1. Working with Linux

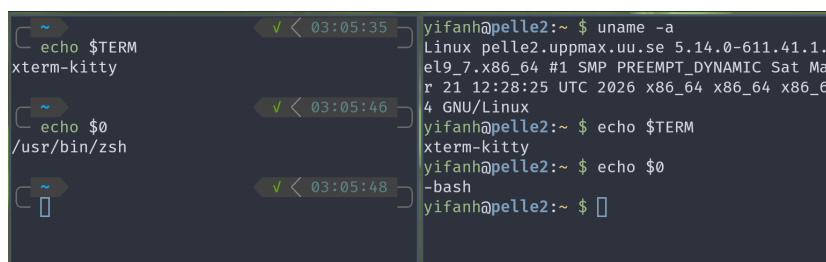
Linux is not a specific system, but a family of systems based on the Linux **kernel**. A specific system from the family is called a Linux **distribution**, or **distro** for short. The distro runs on Futurum is [Ubuntu](#), while for UPPMAX, it becomes [Rocky Linux](#). Different distributions provide different features, but the concepts are shared.

When it comes to ‘working with Linux’, we usually mean to work using CLI specifically.

1.1.1. Terminal, Console, Shell and CLI

The difference between a terminal and a console might not be so clear, as from the view of a user, what they see from their eyes is an input and an output, both human-readable.

In the Linux world, in short, a **terminal** is a wrapper programme, and a **shell** can be run inside. For example, the *Windows Terminal* application is a terminal, which allows the shell *PowerShell* or *cmd* running inside, while the shell can also run standalone.



```
~ 03:05:35 yifanh@pelle2:~ $ uname -a
Linux pelle2.uppmax.uu.se 5.14.0-611.41.1.el9_7.x86_64 #1 SMP PREEMPT_DYNAMIC Sat Mar 21 12:28:25 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
~ 03:05:46 yifanh@pelle2:~ $ echo $TERM
xterm-kitty
~ 03:05:48 yifanh@pelle2:~ $ echo $0
-bash
~ yifanh@pelle2:~ $
```

Figure 1: Outputs of `$TERM` (terminal name) and `$0` (current shell) from local and UPPMAX

A **console** can be considered a special type of terminal. As you can imagine from a gaming console like [Famicon](#), it refers to something physical, and can communicate directly with the machine. So, under these definitions, a console is a piece of hardware physically connected

to the host, while a terminal can connect to the host remotely¹.

A **command-line interface (CLI)** provides a way to interact with software by text. Compared with a **graphical user interface (GUI)**, it requires fewer system resources, and allows users to access utilities easier and faster. However, the advantage of efficiency highly relies on the experience of users. Most of the time, when we meet a new command, the manual and help page are necessary to understand the command and its options.

Table 1 lists some of the commands that may useful for LT students. It is not meaningful to list all of them, as we don't need to be the sysadmin. Clicking the commands will lead you to their tldr help pages, where you can check some common usages.

Common Utilities	Handy Utilities	Text Utilities
cd	df	wc
ls	du	head
cat	file	tail
mkdir	kill	grep
rm	less	sort
cp	tee	uniq
mv	touch	cut
find	pwd	split

Table 1: Some useful commands

Tip

If you want to know the usage of a command, you can always use [man](#) to read the manual page, or search the command at [tldr.sh](#) if you feel it too long.

1.1.2. Users, Groups and Permission

In theory, you don't need to know this part, but as we share the same system, it is better to have such concepts. In short, a **group** is a collection of **users** that share common **permissions** to files, directories, or resources.

In UPPMAX system, this concept of groups is implied by the so-called **Projects**, which means that after you get permitted into a project, you should be added to the related group, allowing you to work in the project folder, but not other projects. You can join and manage projects at the [SUPR Projects](#) page.

Generally, you can use `ls -l` to show files with extra info, including permissions, owner, size, and last-modified timestamp.

```
yifanh@pelle1:/proj/uppmx2025-2-505/woof $ ls -l
total 16
drwxr-sr-x  3 yifanh uppmx2025-2-505 4096 Apr 18 20:11 backup
drwxr-sr-x  4 yifanh uppmx2025-2-505 4096 May  4 20:44 cantonese_daily
drwxr-sr-x 12 yifanh uppmx2025-2-505 4096 Mar 30 18:40 coqui-ai-TTS
-rw-r--r--  1 yifanh uppmx2025-2-505 3814 Apr 19 20:26 requirement.txt
```

Figure 2: Output of `ls -l`

```
yifanh@pelle1:/proj/uppmx2025-2-505/woof $ ls -lah
total 32K
drwxr-sr-x  7 yifanh uppmx2025-2-505 4.0K May  4 23:22 .
drwxrwsr-x 13 root   uppmx2025-2-505 4.0K Apr  6 19:52 ..
drwxr-sr-x  3 yifanh uppmx2025-2-505 4.0K Apr 18 20:11 backup
drwxr-sr-x  4 yifanh uppmx2025-2-505 4.0K May  4 20:44 cantonese_daily
drwxr-sr-x 12 yifanh uppmx2025-2-505 4.0K Mar 30 18:40 coqui-ai-TTS
drwxr-sr-x  2 yifanh uppmx2025-2-505 4.0K Apr  7 16:49 .ipynb_checkpoints
-rw-r--r--  1 yifanh uppmx2025-2-505 3.8K Apr 19 20:26 requirement.txt
drwxr-sr-x  6 yifanh uppmx2025-2-505 4.0K Apr 18 19:08 .TTS_venv
```

Figure 3: Output of `ls -lah`

¹These confusing and messy concepts date back to the time while people still using [teletype](#). If you are interested in this topic, I suggest [this article](#) about TTYs, and [the guide](#) for more general concepts about terminals and consoles.

Tip

If you want to list all files including those hidden, you can use `ls -a` ;
if you want to list files with extra info with human-readable sizes, use `ls -lh` ;
if you want to sort the listed files by last access time (default alphabetical), use `ls -t` ;
you can reverse the order by using `-r`.

Let's take a look to the output

```
-rw-r--r--  1 yifanh uppmx2025-2-505 3.8K Apr 19 20:26 requirement.txt
drwxr-sr-x  6 yifanh uppmx2025-2-505 4.0K Apr 18 19:08 .TTS_venv
```

We have the permissions first, which can be separated into four parts.

0. `r` = read (4), `w` = write (2), `x` = execution (1), (`s` is a special type of `x`, not important for us);
1. The first character indicate the type. `-` means file and `d` means directory;
2. The next nine characters shows security:
 - The first three show the permissions for the owner
 - The next three show the permissions for the group
 - The last three show the permissions for other users (not including `root`²)

Then after that we have the owner `yifanh` and the group `uppmx2025-2-505`. Using the `requirement.txt` file as an example, all these information shows:

1. `x` is not set as a `txt` file usually is not executable;
2. The owner `yifanh` can read and write the file;
3. The user in the group `uppmx2025-2-505` can read the file (yes, you can read other people's file in the same project)
4. For users not in the project, they can read the file also (yes, you probably can try to access a random project and read the files inside)

Sometimes we don't want other people to access our files, then we need to change the permissions with `chmod`.

You might have noticed that I include some numbers for the permissions. If not, it is 4 for read, 2 for write, and 1 for execution. For each part of the permissions, you can add the numbers up to set the permissions you want. For example, the command

```
$ chmod 640 requirement.txt
```

will set the permissions of the file to `rw-` ($4+2+0=6$) `r--` ($4+0+0=4$) `---` ($0+0+0=0$), which removes the read permission for other users compared with the original `644` permissions.

Warning

Avoid setting permissions to `777` (`rw-rw-rw-`) even on you own computer. As this basically gives full permissions to everyone.

²The `root` user is generally immune from the restrictions. They can access any file in the system, but no execution permission if not set.

1.1.3. Editors

Note

Unlike *Rackham* and *Snowy*, VSCode is available in *Pelle* according to [this guide](#) on UPPMAX documentation, but this instruction will not cover this.

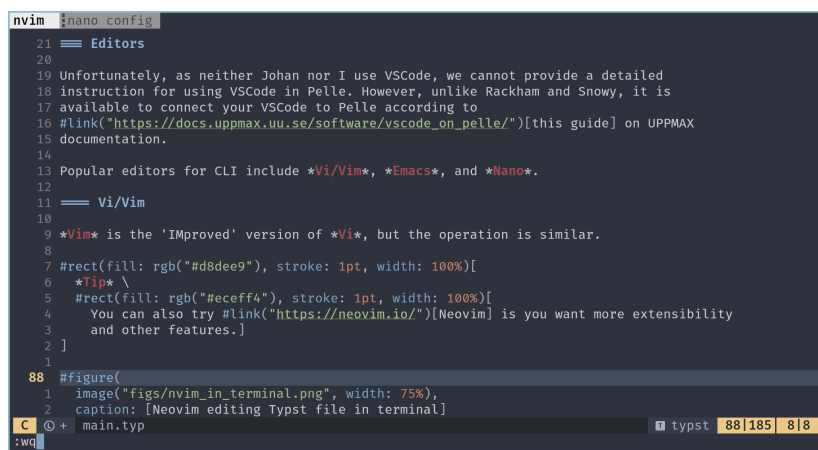
Popular editors for CLI include **Vi/Vim**, **Emacs**, and **Nano**.

1.1.3.1. Vi/Vim

One of the main characters of the editor war. Vim stands for **Vi IMproved**, and you can also try [Neovim](#) or other Vi-like editor if you want some other features, but most of them share the same basic operations.

Note

- Under **Normal mode**, you can move around by pressing `h j k l` or `← ↓ ↑ →`.
- Press `a` or `i` will switch the editor to **Edit mode**, where you can start typing.
- Press `Esc` in Edit mode will bring you back to Normal mode.
- Save the file and quit the editor by pressing `:wq` in Normal mode
- Go to [Vim cheatsheet](#) for more!



```
nvim nano_config
21 == Editors
20
19 Unfortunately, as neither Johan nor I use VSCode, we cannot provide a detailed
18 instruction for using VSCode in Pelle. However, unlike Rackham and Snowy, it is
17 available to connect your VSCode to Pelle according to
16 #link("https://docs.uppmx.uu.se/software/vscode_on_pelle/")[this guide] on UPPMAX
15 documentation.
14
13 Popular editors for CLI include *Vi/Vim*, *Emacs*, and *Nano*.
12
11 == Vi/Vim
10
9 *Vim* is the 'IMproved' version of *Vi*, but the operation is similar.
8
7 #rect(fill: rgb("#d8dee9"), stroke: 1pt, width: 100%)[
6 *Tip* \
5 #rect(fill: rgb("#e6eff4"), stroke: 1pt, width: 100%)[
4 You can also try #link("https://neovim.io/")[Neovim] is you want more extensibility
3 and other features.]
2 ]
1
88 #figure(
1 image("figs/nvim_in_terminal.png", width: 75%),
2 caption: [Neovim editing Typst file in terminal]
C @ + main.typ  typst 88|185 8|8
:wq
```

Figure 4: Neovim in terminal

1.1.3.2. Emacs

Emacs is another main character of the editor war. Usually Emacs has its own interface as Figure 5, but it can be run inside a terminal as Figure 6 also. Although Emacs has commands also, its editing style will be more relied on shortcuts.

Note

- **C** means the **Control/Ctrl** key and **M** means the **Meta** key, which becomes **Alt/Option** for common keyboards nowadays.
- Type **M-x** to input command
- Use **C-g** for aborting a command
- Quit Emacs with **C-x C-c**
- GNU has a [pdf cheatsheet](#) for Emacs

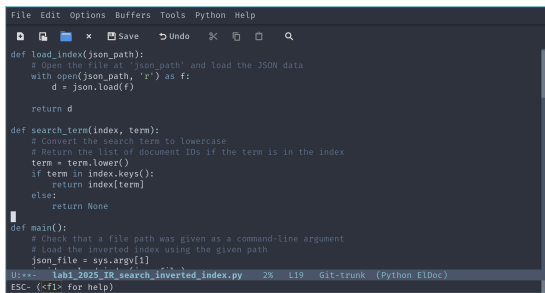


Figure 5: Emacs with its default window

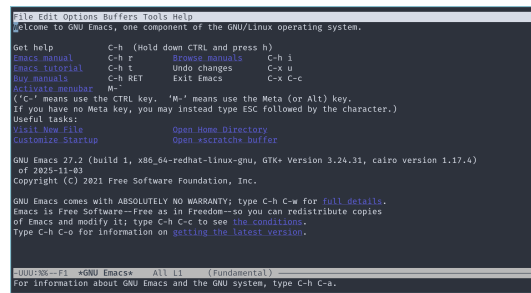


Figure 6: Emacs running inside a terminal

1.1.3.3. Nano

Nano is a light-weight editor originated from **Pico**, which is still available on macOS. It is considered easier to use as it prints some common shortcuts by default. Its light-weight design determines that it will not be as powerful as Vim and Emacs, but some people like using it for small editing.

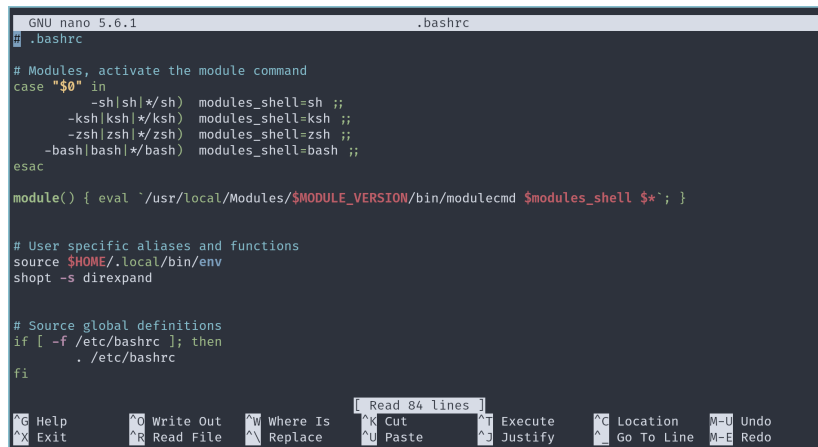


Figure 7: Editing a file with Nano in terminal

Note

Some people might feel these hints not obvious. These symbols in Nano mean:

- **M** is the **Meta** key as said above
- The **^** means the **Control\Ctrl** key, so **^X** means you can exit Nano by pressing **C-x**
- Despit its simplicity, Nano does have a [cheatsheet](#) for all shortcuts

1.2. UPPMAX

Although *Pelle* is the only available cluster to LT students, it is still worth to mention that some concepts from the old handbook, like the chapter about memory, can still be applied despite the design differences between *Pelle* and *Rackham/Snowy*. They are general enough that can be applied to *Futurum*, and even to your future works.

As for UPPMAX itself, just a reminder, don't forget the [UPPMAX documentation](#) for guides for all functions of UPPMAX, and the [system usage chart](#) shows the current system load for each cluster.

1.3. The new cluster: Pelle

1.3.1. What's New

As we all know, the old clusters, *Rackham* and *Snowy*, were [retired](#) earlier this year, so the old days, when we would usually log in to *Rackham* and send heavy or long-time job to *Snowy*, were gone.

For students from Stockholm, besides UPPMAX, you may also try [Dardel](#), which is a similar system maintained by KTH. However, *Dardel* has a slightly different design, so it would be better to refer to [their documentation](#) for more specific information.

1.3.2. Hardwares

As mentioned above, the *Haswell* nodes are from the old *Snowy*. It is suggested to use them as a backup when all other nodes are occupied³.

- CPUs
 - AMD EPYC 9454P 48-Core Processor 2.75 GHz
 - AMD EPYC 9124 16-Core Processor 3 GHz
 - (Haswell) Intel Xeon E5-2630 v3 2.4 GHz
- GPUs
 - Nvidia L40s (48GB)
 - Nvidia H100 (94GB)
 - (Haswell) Nvidia T4
- Memory + Scratch⁴
 - (CPU Node) 768 GiB + 1.7 TiB
 - (Fat Node) 2 TiB + 6.9 TiB
 - (Fat Node) 3 TiB + 6.9 TiB
 - (GPU Node) 384 GiB + 6.9 TiB
 - (Haswell) 256 GiB + 1.8 TiB

³But sometimes even nodes on *Haswell* would be all occupied also.

⁴Scratch is a temporary storage which will be revoked after your job gets done. We will talk about this in Section 7.

2. Moving around

2.1. Login to Pelle

There are several ways to log in to *Pelle*. Similar to *Futurum*, you can use `ssh` or *ThinLinc* client. However, specially, you can use a [remote desktop via website](#) to log in UPPMAX.

No matter which method you choose, you need to set up **two factor authentication (2FA)** to use UPPMAX.

2.1.1. Setting up TOTP

The purpose of 2FA is to get a **Time-based One-Time Password**, or **TOTP** for short. TOTP expires every 30 seconds, and it is not sent by email. You need an application for generating such a code.

At this point, you should already have an account to log in to [SUPR](#), and applied some projects provided by the courses like IR or MT. [Request Uppmax Second Factor](#)

2.1.2. SSH key

Using [SSH keys](#) is considered a more secure method for connection generally, as long as you can keep your keys safe. Specifically for UPPMAX, you can skip entering password and TOTP by using your SSH key.

Warning

The key generation processes below should be processed **on the client**, which in this case, is **your own computer**. Even the server should **NOT** have your private key.

2.1.2.1. Creating the key

The default options for `ssh-keygen` should be enough for normal users. However, it is recommended to use Ed25519 for generating keys, as the command below, unless you have a super old computer. I personally suggest setting a different filename for distinguish, and set a passphrase for unlock the key.

```
$ ssh-keygen -t ed25519 -C "<Comment on the key>"
```

```
~ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kunihiro/.ssh/id_ed25519): test_ed25519
Enter passphrase for "test_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in test_ed25519
Your public key has been saved in test_ed25519.pub
The key fingerprint is:
SHA256:e2p6uT205VY8fh07w/Am7U06h0v2b+yIyv9nPGL02qc kunihiro@fedora-x1c
The key's randomart image is:
+--[ED25519 256]--+
|
|      S      .
| .. .. +o
| .o.+ =o++
| +=o BBo#0|
| .+o.++E600|
+--[SHA256]-----+
```

Figure 8: Generating a key located in `./test_ed25519`

`ssh-keygen` will generate two files for you: one ends with `.pub` is the **public key** and another one is the **private key**. Figure 8 here is only for demonstration, and the key had been revoked after finishing the instruction, but for you, **DO NOT LEAK YOUR PRIVATE KEY**.

2.1.2.2. Upload the key

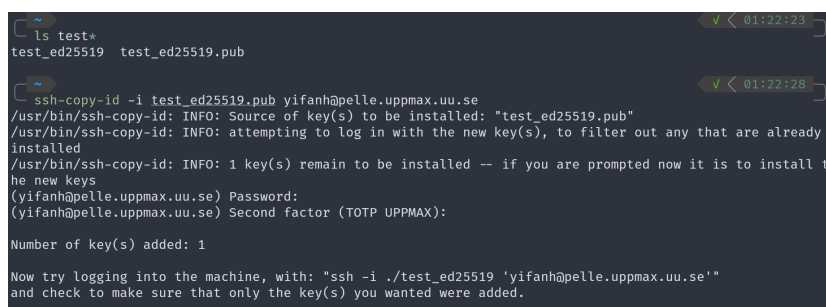
We need to upload the public key to UPPMAX. The easiest way is to use `ssh-copy-id`. Replace `<PATH_OF_PUB_KEY>` to the actual path of your public key.

```
$ ssh-copy-id -i <PATH_OF_PUB_KEY> <USERNAME>@pelle.uppmx.uu.se
```

For Windows, there is no `ssh-copy-id`, so one has to use a slightly weird method: read the contents of the public key, and pipe it to `ssh` to send it to server.

```
PS> cat <PUB_KEY> | ssh <USERNAME>@pelle.uppmx.uu.se "mkdir -p .ssh && tee .ssh/authorized_keys"
```

This command should work for Windows 10 and newer versions.



```
~
ls test*
test_ed25519  test_ed25519.pub

~
ssh-copy-id -i test_ed25519.pub yifanh@pelle.uppmx.uu.se
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "test_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
(yifanh@pelle.uppmx.uu.se) Password:
(yifanh@pelle.uppmx.uu.se) Second factor (TOTP UPPMAX):

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i ./test_ed25519 'yifanh@pelle.uppmx.uu.se'"
and check to make sure that only the key(s) you wanted were added.
```

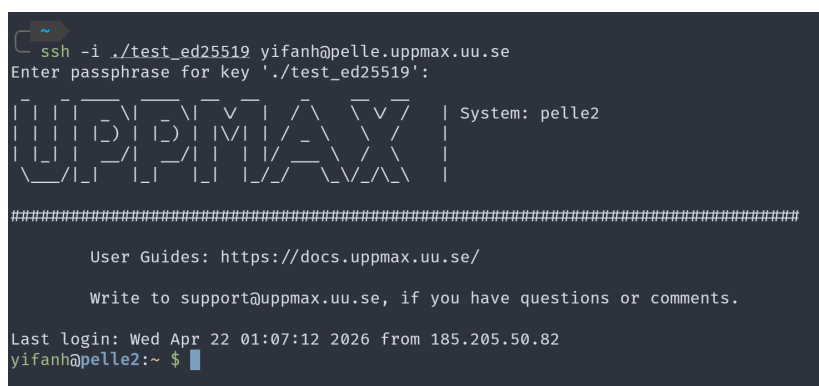
Figure 9: Uploading the key to UPPMAX

2.1.2.3. Login with key

Now you can log in to UPPMAX by specifying your private key. Replace `<PATH_OF_PRIV_KEY>` and `<USERNAME>` to your private key path and your username for login.

```
$ ssh -i <PATH_OF_PRIV_KEY> <USERNAME>@pelle.uppmx.uu.se
```

As Figure 10, will be prompted to input the passphrase of the key, but not the password and TOTP for UPPMAX.



```
~
ssh -i ./test_ed25519 yifanh@pelle.uppmx.uu.se
Enter passphrase for key './test_ed25519':

#####
User Guides: https://docs.uppmx.uu.se/

Write to support@uppmx.uu.se, if you have questions or comments.

Last login: Wed Apr 22 01:07:12 2026 from 185.205.50.82
yifanh@pelle2:~ $
```

Figure 10: Using the key to log in UPPMAX

2.1.2.4. SSH config

By adding the key to config file located in `.ssh/config`, one can login with the provided key by default.

```
Host *.uppmax.uu.se
  IdentityFile <PATH_OF_KEY>
```

```
cat .ssh/config
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/github_ed25519

Host *.uppmax.uu.se
  IdentityFile ~/.ssh/uppmax_ed25519

ssh yifan@pelle.uppmax.uu.se

UPPMAX | System: pelle1
=====
User Guides: https://docs.uppmax.uu.se/
Write to support@uppmax.uu.se, if you have questions or comments.
Last login: Wed Apr 22 01:08:02 2026 from 185.205.50.82
yifan@pelle1: ~
```

Figure 11: SSH config example

2.2. Projects on UPPMAX

Projects

Active Projects You Belong To

Click on the project name in the leftmost column to see more project information.

Project	PI	Project Title	Project Type	Centre	Start Date	End Date	Your Role
UPPMAX 2026/1-95	Sara Stymne	Computational Linguistics @ UU	Centre Local	UPPMAX	2026-04-01	2027-04-01	member
UPPMAX 2026/1-123	Meriem Beloucif	SLN711 & SLN718: Machine Translation x 2	Centre Local	UPPMAX	2026-04-08	2027-01-01	member
UPPMAX 2025/2-505	Eva Pettersson	SLN709 Master's Thesis in Langua	Centre Local Compute	UPPMAX	2026-01-01	2026-12-01	member
UPPMAX 2025/3-5	Sara Stymne	SLN71X - Masterprogram i språkteknologi	Centre Local Compute	UPPMAX	2025-03-25	2026-05-01	member

Request Membership in Project

To request membership in a project, use the search function below to search for the project. You may use project name/Dnr, project title or the name of the principal investigator.

Past Project You Belonged To

Click on the project name in the leftmost column to see more project information.

Project	PI	Project Title	Project Type	Centre	Start Date	End Date	Your Role
UPPMAX 2020/2-2	Joakim Nivre	CL@UU	Centre Local Compute	UPPMAX	2020-03-27	2026-02-01	member

Figure 12: Projects page of SUPR

3. Files Transfer and Synchronisation

3.1. scp

In case you forget, you can always use `scp` for transferring files between the client and the server.

```
$ scp <SOURCE> <DESTINATION>
```

If you want to transfer a folder, use `-r` option like what we normally do for `cp`.

```
$ scp -r <SOURCE> <DESTINATION>
```

For example, you want to upload a file call `model.pth` to a folder called `my_models` under your UPPMAX home directory (So it is `~/my_models/`), then you want to move the folder back to the current directory of your local machine, you can do this on your machine:

- To upload, do:

```
$ scp my_model.pth <USERNAME>@pelle.uppmx.uu.se:my_models
```

- To download, do (don't forget the dot in the end, which means current directory):

```
$ scp -r <USERNAME>@pelle.uppmx.uu.se:my_models .
```

3.2. rsync

Note

Windows does NOT ship `rsync` by default, so you need to have either [WSL](#) or [MinGW](#) or [Cygwin](#) installed

`rsync` is a utility for transferring and synchronising files like `scp`, but it provides more advanced features like excluding files, reduces latency and so on.

`rsync` can be used for transferring files like `rsync source target` as `scp`, but more commonly it is used for synchronising, so the basic operations are more related to a directory like:

- `$ rsync -a source_dir target_dir` for copying a directory, and
- `$ rsync -a source_dir/ target_dir` for all contents in the directory.

Warning

Pay attention to the final slash when using `rsync`.

- Without the slash, the folder itself will be included for transferring
- With the slash, the folder will not be included. Instead, you will transfer all your files in the folder

If you are not sure about what you typed, use options `-anv` for a dry-run and verbose output.

Using the same example above, we have:

- To synchronise all contents from a local folder called `my_models/` to server, do:

```
$ rsync -av my_models/ <USERNAME>@pelle.uppmax.uu.se:my_models
```

- To do it in opposite direction but with the directory also:

```
$ rsync -av <USERNAME>@pelle.uppmax.uu.se:my_models .
```

4. Softwares

Usually for Linux, there should be a package manager available to install new softwares. However, as we don't have the permission for such tasks, we have to rely on other methods to install softwares.

4.1. Module System

The module system used by UPPMAX is called [Lmod](#), but for convenience, it is wrapped as the `module` command in UPPMAX. You can find all available modules [here](#)⁵.

- To **search a module**, use `$ module spider <MODULE_NAME>`

Tip

You can search a module with RegEx with the `-r` option as:

```
$ module -r spider <EXPRESSION>
```

- To **load a module**, use `$ module add <MODULE_NAME>`

Note

The system distinguishes the upper cases and the lower cases, which means that when you want to load a module called `FFmpeg`, it will complain if you write `ffmpeg`.

- To **unload all modules**, use `$ module purge`
- To **list loaded modules**, use `$ module list`

4.2. Installing Binary

However, there are still chances that a module is unavailable on the system. While you can send an email to [UPPMAX support](#), you can download the binary and run it locally.

4.3. Compiling

It is a rare situation, but if there is no binary file provided, you can try to compile things yourself.

There is no a common method to build a software, as there are so many toolchains for different usages and different languages. Thus, this session is more like a reminder that this way exists, and you can follow the provided guide (if any) to build the software.

⁵This table list softwares on *Bianca*, but all of them should be included in *Pelle* also.

Building

Configure the build using CMake:

```
cmake -Bbuild -DCMAKE_INSTALL_PREFIX=/usr
```

NOTE: The `-DCMAKE_INSTALL_PREFIX` option above will install the files to the `/usr` directory, instead of the default `/usr/local` location. For more information on available options, you can run:

```
cmake -Bbuild -LH
```

To use a different compiler, or compiler flags, you can specify these before the `cmake` command. For example:

```
CC=clang CFLAGS=-Wextra cmake -Bbuild -DCMAKE_INSTALL_PREFIX=/usr
```

The `espeak-ng` and `speak-ng` programs, along with the `espeak-ng` voices, can then be built with:

```
cmake --build build
```

The data (language dictionaries, phoneme tables, intonation) can be built with:

```
cmake --build build --target data
```

Specific languages can be compiled by running the built binary directly:

```
ESPEAK_DATA_PATH='pwd'/build/build/src/espeak-ng --compile=LANG
```

Figure 13: I once had to compile the software myself following the provided guide.

5. Python Virtual Environment

5.1. python-venv

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

5.2. conda

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

5.3. uv

[uv](#) is one of my favourite tool for doing Python stuff, as most of the time I don't need modified packages from conda-forge, and it is super fast. You can give it a try if conda runs slow.

5.3.1. Installation

uv provides a standalone installer. You don't need a computing node for the installation.

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

5.3.2. Using venv

The management of venv with uv is more similar to the original venv. One can create a venv by running:

```
$ uv venv <VENV_NAME> --python <PY_VER>
```

The Python version is optional. If not set, it will use the latest version. The venv can be activated with the original style:

```
$ source <VENV_NAME>/bin/activate
```

And deactivated with

```
$ deactivate
```

5.3.3. Packages

uv is not intended to be an exact clone of pip, but swapping out `pip install` for `uv pip install` should work most of the time. For example:

```
$ uv pip install torch datasets transformers
```

The `uv pip` inherits most options of the original `pip`. For example, you can get a list of packages by running:

```
$ uv pip freeze
```

6. Scheduling Jobs on Pelle

6.1. Nodes

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

6.2. Interactive Session

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

6.3. Slurm

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

6.4. Time Management for Jobs

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

7. Storage Management

7.1. Available Storages

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

7.2. Storage Usage

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

7.3. Cache

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

8. Using Jupyter Lab or Notebook

8.1. UPPMAX Lab

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

8.2. Running on an Interactive Session

If you just need a temporary space for testing some idea, I strongly suggest using the so-called [UPPMAX Lab](#)⁶. It provides some pre-built environments for running temporary, ephemeral, non-persistent works, as all data will be lost when the environment is deleted, and an environment will expire after two days.

But if you somehow do require a full node to run notebook, here are the steps.

8.2.1. Request an interactive session

Login as usual, and start an interactive session.

```
$ interactive -A <PROJ_NAME> -p <NODE> -t <TIME>
```

Pay attention to the name of node that allocated to you, as we will use the name later. The name starts with 'p' followed by some digits.

8.2.2. Run Notebook in the interactive session

You can also specify a port if necessary by setting `--port=<PORT_NUM>`.

```
$ jupyter notebook --no-browser --ip=0.0.0.0
```

8.2.3. SSH forward

Now open another terminal, connect to UPPMAX with port forwarding option added: `-L <PORT>:<HOST>:<HOSTPORT>`.

`<PORT>` and `<HOSTPORT>` is the one you set before for Jupyter Notebook, by default `8888`, and `<HOST>` is the name of node that mentioned before. For example, if your port is set to `2333` and node is `p201`, you should start a new connection by:

```
$ ssh -L 2333:p201:2333 <USERNAME>@pelle.uppmx.uu.se
```

Then you can click the link with the address `127.0.0.1` from Jupyter to open Notebook from your local browser.

⁶Currently no official name for this service

Note

You should **NOT** terminate any connections until you finish using Notebook.

9. Git

You will probably use Git or related services at some point in the programme, but not sure by how when I write this. Anyway, it would be great to know Git as 96% of professional developers use Git, even though you are going to let AI agents to manage your repositories.

Git is a **distributed version control system** created by Linus Torvalds (yes, the guy who made Linux kernel). In short, it means that using with Git, you can switch between different changes easily, contribute to the project with others parallelly and get notified about conflicts automatically.

Before you start, please ensure that Git is installed in your computer. Sometimes Git has been shipped by default, but if it is not the case, you can Sometimes Git has been shipped by default, but if it is not the case, you can always check [The Git Install Page](#) for details.

9.1. Forges